

# Lab 10 - Interactive Visualization

We recommend downloading this QMD file to use as a template for your answers. You can download the file from: <https://github.com/dmcable/BIOSTAT620W26/blob/main/labs/lab10.qmd>

We have set `eval=FALSE` as a global option.

To run a specific chunk, you can set `eval=TRUE` in that chunk.

To run all chunks, you can set `eval=TRUE` inside of `opts_chunk$set()` in the `setup` chunk.

## Learning Goals

- Read in and process the COVID dataset from the New York Times GitHub repository
- Create interactive graphs of different types using `plot_ly()` and `ggplotly()` functions
- Customize the hoverinfo and other plot features
- Create a Choropleth map using `plot_geo()`

## Lab Description

We will work with COVID data downloaded from the New York Times. The dataset consists of COVID-19 cases and deaths in each US state during the course of the COVID epidemic.

**The objective of this lab is to explore relationships between cases, deaths, and population sizes of US states, and plot data to demonstrate this**

## Steps

### I. Reading and processing the New York Times (NYT) state-level COVID-19 data

---

#### 1. Read in the data

- Read in the COVID data with `read.csv()` from the NYT GitHub repository:  
<https://raw.githubusercontent.com/nytimes/covid-19-data/master/us-states.csv>
- Read in the state population data with `read.csv()` from the repository:  
[https://raw.githubusercontent.com/COVID19Tracking/associated-data/master/us\\_census\\_data/us\\_census\\_2018\\_population\\_estimates\\_states.csv](https://raw.githubusercontent.com/COVID19Tracking/associated-data/master/us_census_data/us_census_2018_population_estimates_states.csv)
- Merge datasets

```
## data extracted from New York Times state-level data from NYT Github repository
# https://github.com/nytimes/covid-19-data
```

```
## state-level population information from us_census_data available on GitHub repository:
# https://github.com/COVID19Tracking/associated-data/tree/master/us_census_data

### FINISH THE CODE HERE ###
# load COVID state-level data from NYT
cv_states <- read_csv('https://raw.githubusercontent.com/nytimes/covid-19-data/refs/heads/master/

### FINISH THE CODE HERE ###
# load state population data
state_pops <- read_csv('https://raw.githubusercontent.com/COVID19Tracking/associated-data/refs/heads/master/')
```

```
# adjust column names
### FINISH THE CODE HERE ###
state_pops$abbs <- state_pops$state
state_pops$state <- state_pops$state_name
state_pops$state_name <- NULL
```

```
### FINISH THE CODE HERE
cv_states <- merge(cv_states, state_pops, by = 'state')
```

## 2. Look at the data

- Inspect the dimensions, `head`, and `tail` of the data
- Inspect the structure of each variables

```
dim(cv_states)
```

```
[1] 58094      9
```

```
head(cv_states)
```

|   | state   | date       | fips | cases   | deaths | geo_id | population | pop_density | abbs |
|---|---------|------------|------|---------|--------|--------|------------|-------------|------|
| 1 | Alabama | 2023-01-04 | 01   | 1587224 | 21263  | 01     | 4887871    | 96.50939    | AL   |
| 2 | Alabama | 2020-04-25 | 01   | 6213    | 213    | 01     | 4887871    | 96.50939    | AL   |
| 3 | Alabama | 2023-02-26 | 01   | 1638348 | 21400  | 01     | 4887871    | 96.50939    | AL   |
| 4 | Alabama | 2022-12-03 | 01   | 1549285 | 21129  | 01     | 4887871    | 96.50939    | AL   |
| 5 | Alabama | 2020-05-06 | 01   | 8691    | 343    | 01     | 4887871    | 96.50939    | AL   |
| 6 | Alabama | 2021-04-21 | 01   | 524367  | 10807  | 01     | 4887871    | 96.50939    | AL   |

```
tail(cv_states)
```

|       | state   | date       | fips | cases  | deaths | geo_id | population | pop_density | abbs |
|-------|---------|------------|------|--------|--------|--------|------------|-------------|------|
| 58089 | Wyoming | 2022-09-11 | 56   | 175290 | 1884   | 56     | 577737     | 5.950611    | WY   |
| 58090 | Wyoming | 2022-08-21 | 56   | 173487 | 1871   | 56     | 577737     | 5.950611    | WY   |
| 58091 | Wyoming | 2021-01-26 | 56   | 51152  | 596    | 56     | 577737     | 5.950611    | WY   |
| 58092 | Wyoming | 2021-02-21 | 56   | 53795  | 662    | 56     | 577737     | 5.950611    | WY   |

```
58093 Wyoming 2021-08-22 56 70671 809 56 577737 5.950611 WY
58094 Wyoming 2021-03-20 56 55581 693 56 577737 5.950611 WY
```

```
str(cv_states)
```

```
'data.frame': 58094 obs. of 9 variables:
 $ state      : chr  "Alabama" "Alabama" "Alabama" "Alabama" ...
 $ date       : Date, format: "2023-01-04" "2020-04-25" ...
 $ fips       : chr  "01" "01" "01" "01" ...
 $ cases      : num  1587224 6213 1638348 1549285 8691 ...
 $ deaths     : num  21263 213 21400 21129 343 ...
 $ geo_id     : chr  "01" "01" "01" "01" ...
 $ population : num  4887871 4887871 4887871 4887871 4887871 ...
 $ pop_density: num  96.5 96.5 96.5 96.5 96.5 ...
 $ abbs       : chr  "AL" "AL" "AL" "AL" ...
```

```
dim(state_pops)
```

```
[1] 52 5
```

```
head(state_pops)
```

```
# A tibble: 6 × 5
  state      geo_id population pop_density abbs
  <chr>      <chr>      <dbl>      <dbl> <chr>
1 Alabama    01          4887871      96.5 AL
2 Alaska     02          737438       1.29 AK
3 Arizona    04          7171646      63.1 AZ
4 Arkansas   05          3013825      57.9 AR
5 California 06          39557045     254. CA
6 Colorado   08          5695564      55.0 CO
```

```
tail(state_pops)
```

```
# A tibble: 6 × 5
  state      geo_id population pop_density abbs
  <chr>      <chr>      <dbl>      <dbl> <chr>
1 Virginia   51          8517685     216. VA
2 Washington 53          7535591     113. WA
3 West Virginia 54          1805832      75.1 WV
4 Wisconsin  55          5813568     107. WI
5 Wyoming    56           577737      5.95 WY
6 Puerto Rico 72          3195153      NA PR
```

```
str(state_pops)
```

```
spc_tbl_ [52 × 5] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ state      : chr [1:52] "Alabama" "Alaska" "Arizona" "Arkansas" ...
```

```
$ geo_id      : chr [1:52] "01" "02" "04" "05" ...
$ population : num [1:52] 4887871 737438 7171646 3013825 39557045 ...
$ pop_density: num [1:52] 96.51 1.29 63.14 57.92 253.91 ...
$ abbs       : chr [1:52] "AL" "AK" "AZ" "AR" ...
- attr(*, "spec")=
.. cols(
..   state = col_character(),
..   state_name = col_character(),
..   geo_id = col_character(),
..   population = col_double(),
..   pop_density = col_double()
.. )
- attr(*, "problems")=<externalptr>
```

### 3. Format the data

- Make date into a date variable
- Make state and state abbreviation (abb) into a factor variables
- Order the data first by state, second by date
- Confirm the variables are now correctly formatted
- Inspect the range values for each variable

```
# format the date
cv_states$date <- as.Date(cv_states$date)

# format the state and state abbreviation (abb) variables
cv_states$state <- factor(cv_states$state)
cv_states$abb <- factor(cv_states$abb)

### FINISH THE CODE HERE
# order the data first by state, second by date
cv_states <- cv_states[order(cv_states$state, cv_states$date),]

# Confirm the variables are now correctly formatted
str(cv_states)
```

```
'data.frame':  58094 obs. of  10 variables:
 $ state      : Factor w/ 52 levels "Alabama","Alaska",...: 1 1 1 1 1 1 1 1 1 1 ...
 $ date       : Date, format: "2020-03-13" "2020-03-14" ...
 $ fips       : chr  "01" "01" "01" "01" ...
 $ cases      : num  6 12 23 29 39 51 78 106 131 157 ...
 $ deaths     : num  0 0 0 0 0 0 0 0 0 0 ...
 $ geo_id     : chr  "01" "01" "01" "01" ...
 $ population : num  4887871 4887871 4887871 4887871 4887871 ...
 $ pop_density: num  96.5 96.5 96.5 96.5 96.5 ...
 $ abbs       : chr  "AL" "AL" "AL" "AL" ...
 $ abb        : Factor w/ 52 levels "AK","AL","AR",...: 2 2 2 2 2 2 2 2 2 2 ...
```

```
head(cv_states)
```



|      | state   | date       | fips | cases | deaths | geo_id | population | pop_density | abbs |
|------|---------|------------|------|-------|--------|--------|------------|-------------|------|
| 1029 | Alabama | 2020-03-13 | 01   | 6     | 0      | 01     | 4887871    | 96.50939    | AL   |
| 597  | Alabama | 2020-03-14 | 01   | 12    | 0      | 01     | 4887871    | 96.50939    | AL   |
| 282  | Alabama | 2020-03-15 | 01   | 23    | 0      | 01     | 4887871    | 96.50939    | AL   |
| 12   | Alabama | 2020-03-16 | 01   | 29    | 0      | 01     | 4887871    | 96.50939    | AL   |
| 266  | Alabama | 2020-03-17 | 01   | 39    | 0      | 01     | 4887871    | 96.50939    | AL   |
| 78   | Alabama | 2020-03-18 | 01   | 51    | 0      | 01     | 4887871    | 96.50939    | AL   |
|      | abb     |            |      |       |        |        |            |             |      |
| 1029 | AL      |            |      |       |        |        |            |             |      |
| 597  | AL      |            |      |       |        |        |            |             |      |
| 282  | AL      |            |      |       |        |        |            |             |      |
| 12   | AL      |            |      |       |        |        |            |             |      |
| 266  | AL      |            |      |       |        |        |            |             |      |
| 78   | AL      |            |      |       |        |        |            |             |      |

```
tail(cv_states)
```

|       | state   | date       | fips | cases  | deaths | geo_id | population | pop_density | abbs |
|-------|---------|------------|------|--------|--------|--------|------------|-------------|------|
| 57902 | Wyoming | 2023-03-18 | 56   | 185640 | 2009   | 56     | 577737     | 5.950611    | WY   |
| 57916 | Wyoming | 2023-03-19 | 56   | 185640 | 2009   | 56     | 577737     | 5.950611    | WY   |
| 57647 | Wyoming | 2023-03-20 | 56   | 185640 | 2009   | 56     | 577737     | 5.950611    | WY   |
| 57867 | Wyoming | 2023-03-21 | 56   | 185800 | 2014   | 56     | 577737     | 5.950611    | WY   |
| 58057 | Wyoming | 2023-03-22 | 56   | 185800 | 2014   | 56     | 577737     | 5.950611    | WY   |
| 57812 | Wyoming | 2023-03-23 | 56   | 185800 | 2014   | 56     | 577737     | 5.950611    | WY   |
|       | abb     |            |      |        |        |        |            |             |      |
| 57902 | WY      |            |      |        |        |        |            |             |      |
| 57916 | WY      |            |      |        |        |        |            |             |      |
| 57647 | WY      |            |      |        |        |        |            |             |      |
| 57867 | WY      |            |      |        |        |        |            |             |      |
| 58057 | WY      |            |      |        |        |        |            |             |      |
| 57812 | WY      |            |      |        |        |        |            |             |      |

```
# Inspect the range values for each variable.
head(cv_states)
```

|      | state   | date       | fips | cases | deaths | geo_id | population | pop_density | abbs |
|------|---------|------------|------|-------|--------|--------|------------|-------------|------|
| 1029 | Alabama | 2020-03-13 | 01   | 6     | 0      | 01     | 4887871    | 96.50939    | AL   |
| 597  | Alabama | 2020-03-14 | 01   | 12    | 0      | 01     | 4887871    | 96.50939    | AL   |
| 282  | Alabama | 2020-03-15 | 01   | 23    | 0      | 01     | 4887871    | 96.50939    | AL   |
| 12   | Alabama | 2020-03-16 | 01   | 29    | 0      | 01     | 4887871    | 96.50939    | AL   |
| 266  | Alabama | 2020-03-17 | 01   | 39    | 0      | 01     | 4887871    | 96.50939    | AL   |
| 78   | Alabama | 2020-03-18 | 01   | 51    | 0      | 01     | 4887871    | 96.50939    | AL   |
|      | abb     |            |      |       |        |        |            |             |      |
| 1029 | AL      |            |      |       |        |        |            |             |      |
| 597  | AL      |            |      |       |        |        |            |             |      |
| 282  | AL      |            |      |       |        |        |            |             |      |
| 12   | AL      |            |      |       |        |        |            |             |      |
| 266  | AL      |            |      |       |        |        |            |             |      |
| 78   | AL      |            |      |       |        |        |            |             |      |

```
summary(cv_states)
```

```

      state      date      fips
Washington : 1158   Min.   :2020-01-21   Length:58094
Illinois   : 1155   1st Qu.:2020-12-06   Class :character
California : 1154   Median :2021-09-11   Mode  :character
Arizona    : 1153   Mean   :2021-09-10
Massachusetts: 1147   3rd Qu.:2022-06-17
Wisconsin   : 1143   Max.   :2023-03-23
(Other)    :51184

      cases      deaths      geo_id      population
Min.   :      1   Min.   :      0   Length:58094   Min.   : 577737
1st Qu.: 112125   1st Qu.: 1598   Class :character   1st Qu.: 1805832
Median : 418121   Median : 5901   Mode  :character   Median : 4468402
Mean   : 947941   Mean   : 12553                Mean   : 6397965
3rd Qu.: 1134318   3rd Qu.: 15952                3rd Qu.: 7535591
Max.   :12169158   Max.   :104277                Max.   :39557045

      pop_density      abbs      abb
Min.   :      1.292   Length:58094   WA      : 1158
1st Qu.: 43.659      Class :character   IL      : 1155
Median : 107.860      Mode  :character   CA      : 1154
Mean   : 423.031                AZ      : 1153
3rd Qu.: 229.511                MA      : 1147
Max.   :11490.120                WI      : 1143
NA's   :1106                    (Other):51184

```

```
min(cv_states$date)
```

```
[1] "2020-01-21"
```

```
max(cv_states$date)
```

```
[1] "2023-03-23"
```

## 4. Add `new_cases` and `new_deaths` and correct outliers

- Add variables for new cases, `new_cases`, and new deaths, `new_deaths`:
  - Hint: You can set `new_cases` equal to the difference between cases on date *i* and date *i*-1, starting on date *i*=2
- Filter to dates after June 1, 2021
- Use `plotly` for EDA: See if there are outliers or values that don't make sense for `new_cases` and `new_deaths`. Which states and which dates have strange values?
- Correct outliers: Set negative values for `new_cases` or `new_deaths` to 0

- Recalculate `cases` and `deaths` as cumulative sum of updated `new_cases` and `new_deaths`
- Get the rolling average of new cases and new deaths to smooth over time
- Inspect data again interactively

```
state_list <- unique(cv_states$state)
state_list
```

```
[1] Alabama      Alaska      Arizona
[4] Arkansas     California  Colorado
[7] Connecticut  Delaware    District of Columbia
[10] Florida      Georgia     Hawaii
[13] Idaho        Illinois    Indiana
[16] Iowa         Kansas      Kentucky
[19] Louisiana    Maine       Maryland
[22] Massachusetts Michigan    Minnesota
[25] Mississippi  Missouri   Montana
[28] Nebraska     Nevada     New Hampshire
[31] New Jersey   New Mexico  New York
[34] North Carolina North Dakota Ohio
[37] Oklahoma     Oregon      Pennsylvania
[40] Puerto Rico  Rhode Island South Carolina
[43] South Dakota Tennessee    Texas
[46] Utah         Vermont     Virginia
[49] Washington   West Virginia Wisconsin
[52] Wyoming

52 Levels: Alabama Alaska Arizona Arkansas California Colorado ... Wyoming
```

```
# Add variables for new_cases and new_deaths:
for (i in 1:length(state_list)) {
  cv_subset <- subset(cv_states, state == state_list[i])
  cv_subset <- cv_subset[order(cv_subset$date),]

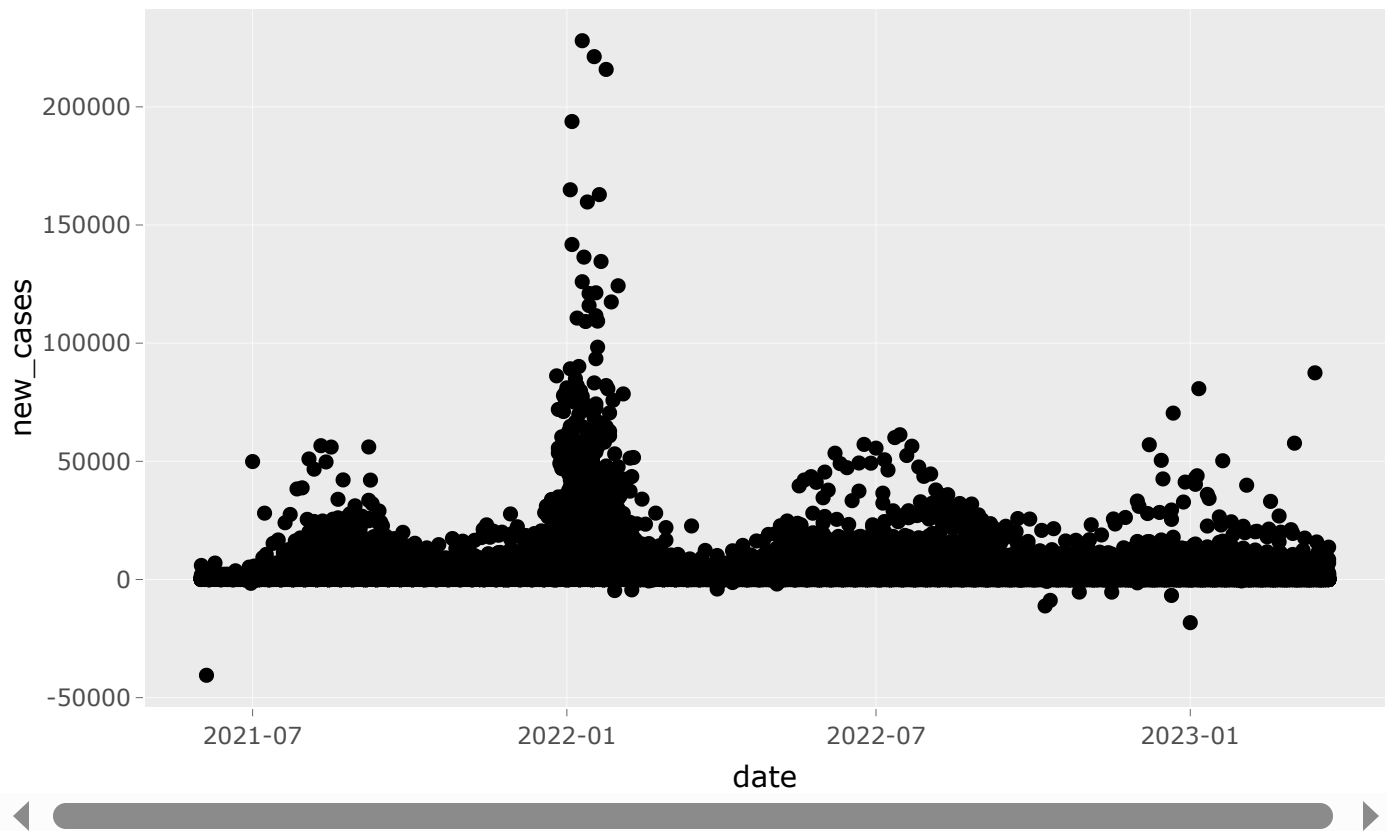
  # add starting level for new cases and deaths
  cv_subset$new_cases <- cv_subset$cases[1]
  cv_subset$new_deaths <- cv_subset$deaths[1]

  ### FINISH THE CODE HERE
  for (j in 2:nrow(cv_subset)) {
    cv_subset$new_cases[j] <- cv_subset$cases[j] - cv_subset$cases[j - 1]
    cv_subset$new_deaths[j] <- cv_subset$deaths[j] - cv_subset$deaths[j - 1]
  }

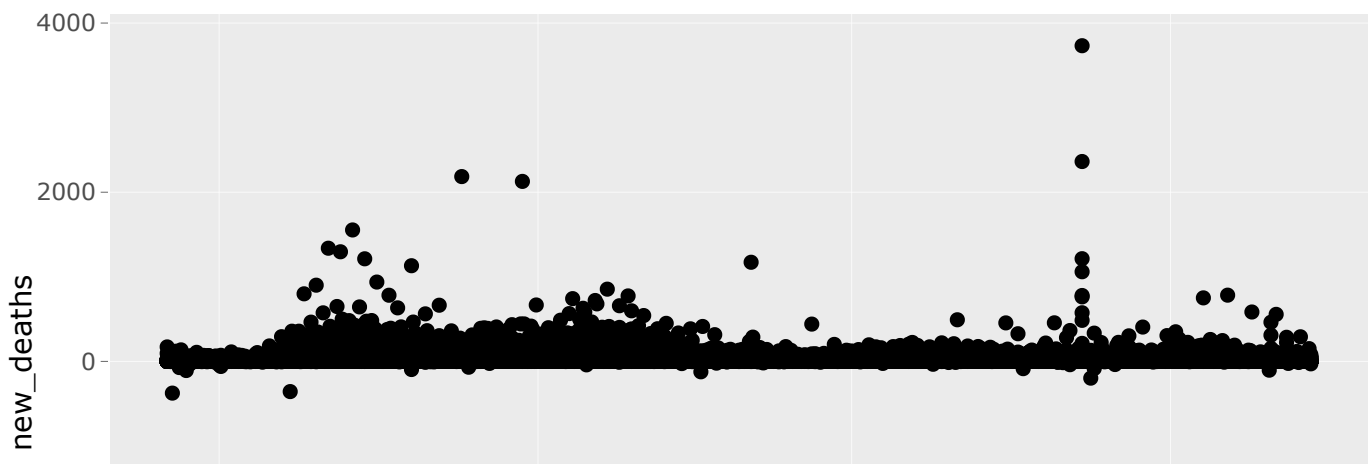
  # include in main dataset
  cv_states$new_cases[cv_states$state==state_list[i]] <- cv_subset$new_cases
  cv_states$new_deaths[cv_states$state==state_list[i]] <- cv_subset$new_deaths
}
```

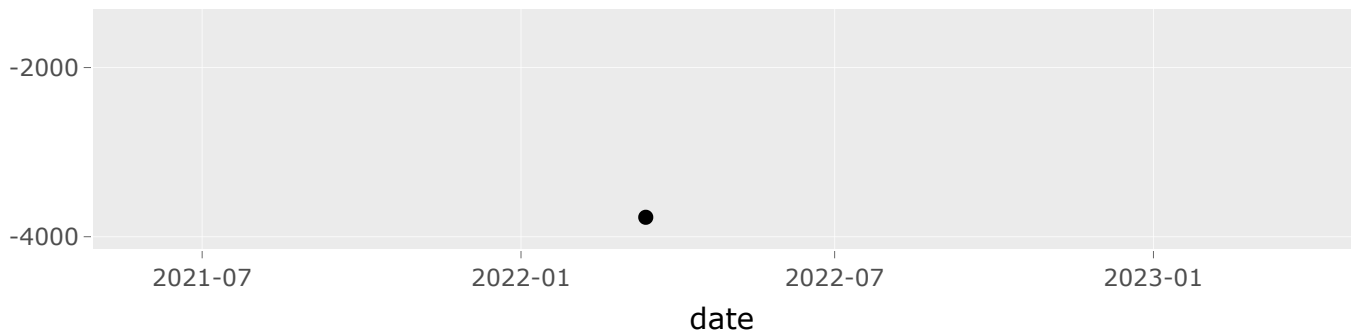
```
# Focus on recent dates  
cv_states <- cv_states |> dplyr::filter(date >= "2021-06-01")
```

```
# Inspect outliers in new_cases using plotly  
#### COMPLETE CODE HERE  
p1 <- ggplot(cv_states, aes(date, new_cases)) +  
  geom_point()  
ggplotly(p1)
```



```
p2 <- ggplot(cv_states, aes(date, new_deaths)) +  
  geom_point()  
ggplotly(p2)
```





```
# set negative new case or death counts to 0
cv_states$new_cases[cv_states$new_cases<0] <- 0
cv_states$new_deaths[cv_states$new_deaths<0] <- 0

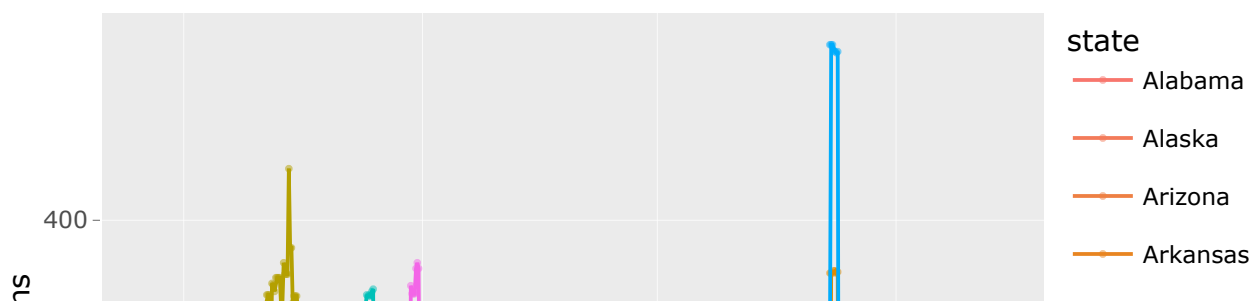
# Re-calculate `cases` and `deaths` as cumulative sum of updated `new_cases` and `new_deaths`
for (i in 1:length(state_list)) {
  cv_subset = subset(cv_states, state == state_list[i])

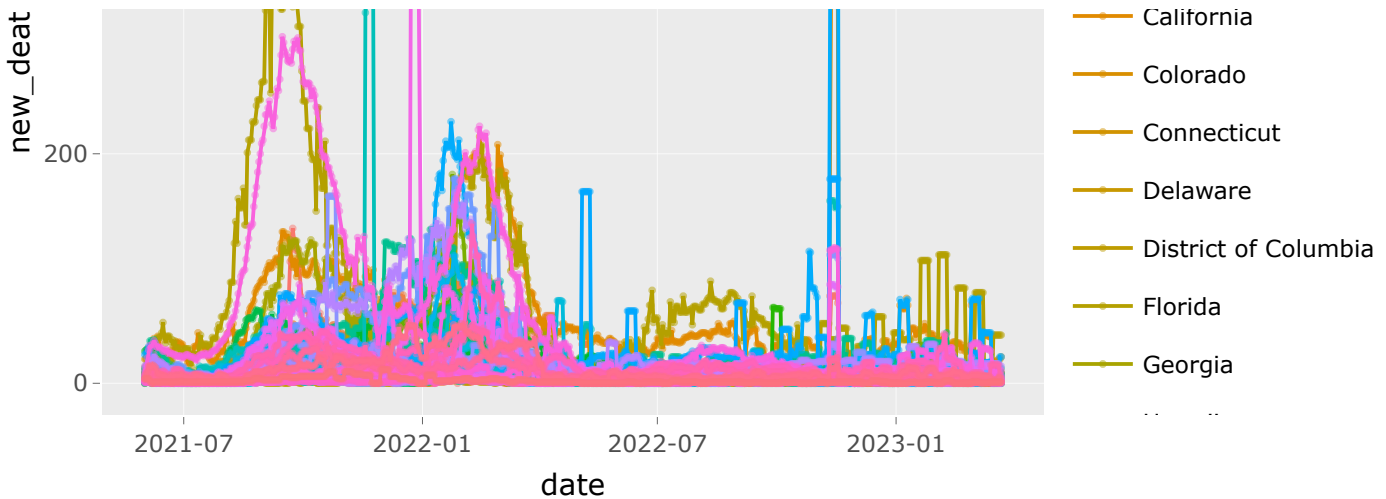
  # add starting level for new cases and deaths
  cv_subset$cases <- cv_subset$cases[1]
  cv_subset$deaths <- cv_subset$deaths[1]

  ### FINISH CODE HERE
  for (j in 2:nrow(cv_subset)) {
    cv_subset$cases[j] <- cv_subset$new_cases[j] + cv_subset$cases[j - 1]
    cv_subset$deaths[j] <- cv_subset$new_deaths[j] + cv_subset$deaths[j - 1]
  }
  # include in main dataset
  cv_states$cases[cv_states$state==state_list[i]] <- cv_subset$cases
  cv_states$deaths[cv_states$state==state_list[i]] <- cv_subset$deaths
}

# Smooth new counts
cv_states$new_cases <- zoo::rollmean(cv_states$new_cases, k=7, fill=NA, align='right') |> round(d
cv_states$new_deaths <- zoo::rollmean(cv_states$new_deaths, k=7, fill=NA, align='right') |> round

# Inspect data again interactively
p2 <- ggplot(cv_states, aes(x = date, y = new_deaths, color = state)) + geom_line() + geom_point()
ggplotly(p2)
```





## 5. Add additional variables

- Add population-normalized (by 100,000) variables for each variable type (rounded to 1 decimal place). Make sure the variables you calculate are in the correct format (`numeric`). You can use the following variable names:

- `per100k` = cases per 100,000 population
- `newper100k` = new cases per 100,000
- `deathspers100k` = deaths per 100,000
- `newdeathspers100k` = new deaths per 100,000

- Add a `naive_CFR` variable representing `deaths / cases` on each date for each state
- Create a data frame representing values on the most recent date, `cv_states_today`, as done in lecture

```
### FINISH CODE HERE
# add population normalized (by 100,000) counts for each variable
cv_states$per100k <- as.numeric(format(round(cv_states$cases/(cv_states$population/100000),1),nsm
cv_states$newper100k <- as.numeric(format(round(cv_states$new_cases/(cv_states$population/100000)
cv_states$deathspers100k <- as.numeric(format(round(cv_states$deaths/(cv_states$population/100000)
cv_states$newdeathspers100k <- as.numeric(format(round(cv_states$new_deaths/(cv_states$population/100000)

# add a naive_CFR variable = deaths / cases
cv_states <- cv_states |> mutate(naive_CFR = round((cv_states$deaths / cv_states$cases),2))

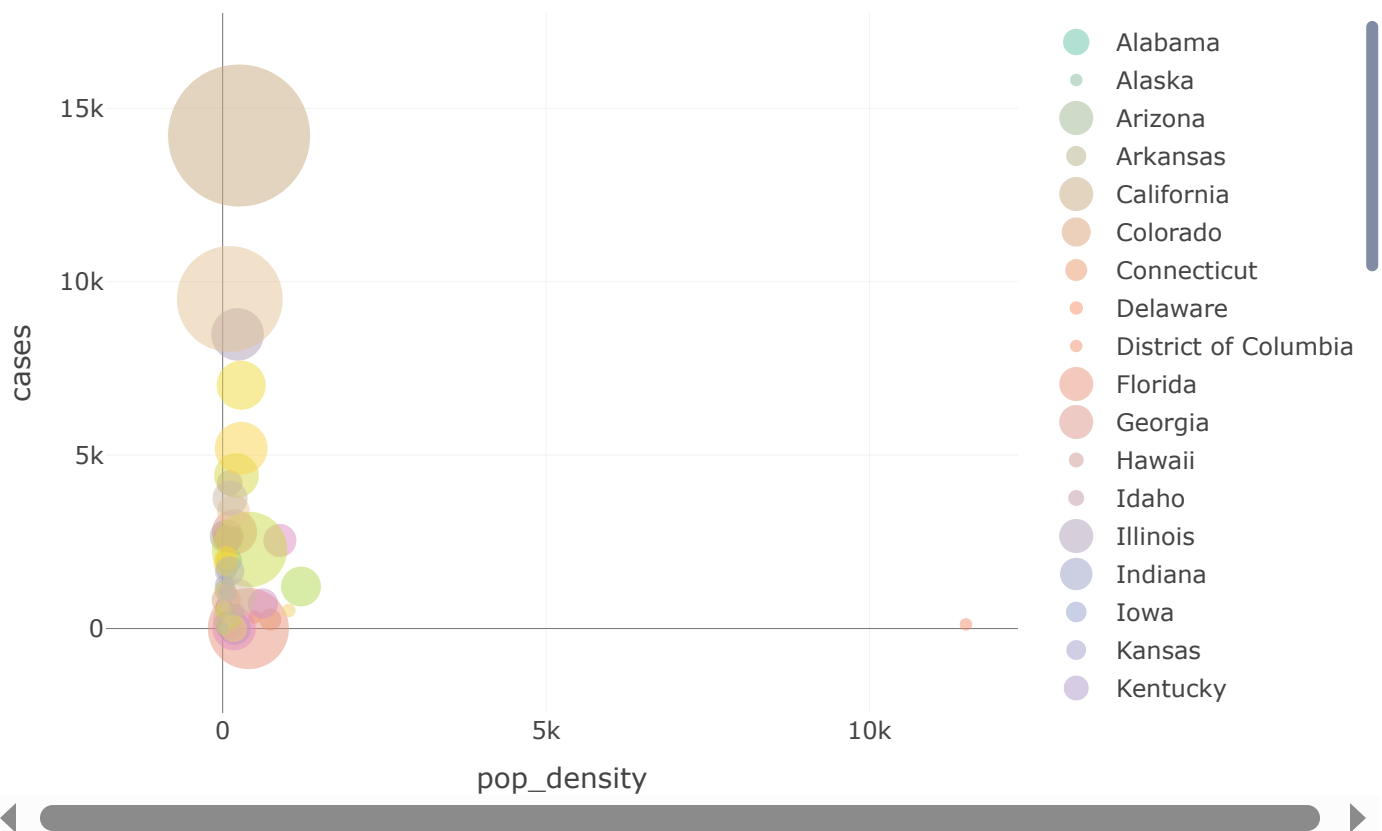
# create a `cv_states_today` variable
cv_states_today <- subset(cv_states, date==max(cv_states$date))
```

## II. Scatterplots

### 6. Explore scatterplots using `plot_ly()`

- Create a scatterplot using `plot_ly()` representing `pop_density` vs. various variables (e.g. `cases`, `per100k`, `deaths`, `deathsper100k`) for each state on most recent date (`cv_states_today`)
  - Color points by state and size points by state population
  - Use hover to identify any outliers.
  - Remove those outliers and replot.
- Choose one plot. For this plot:
  - Add `hoverinfo` specifying the state name, cases per 100k, and deaths per 100k, similarly to how we did this in the lecture notes
  - Add layout information to title the chart and the axes
  - Enable `hovermode = "compare"`

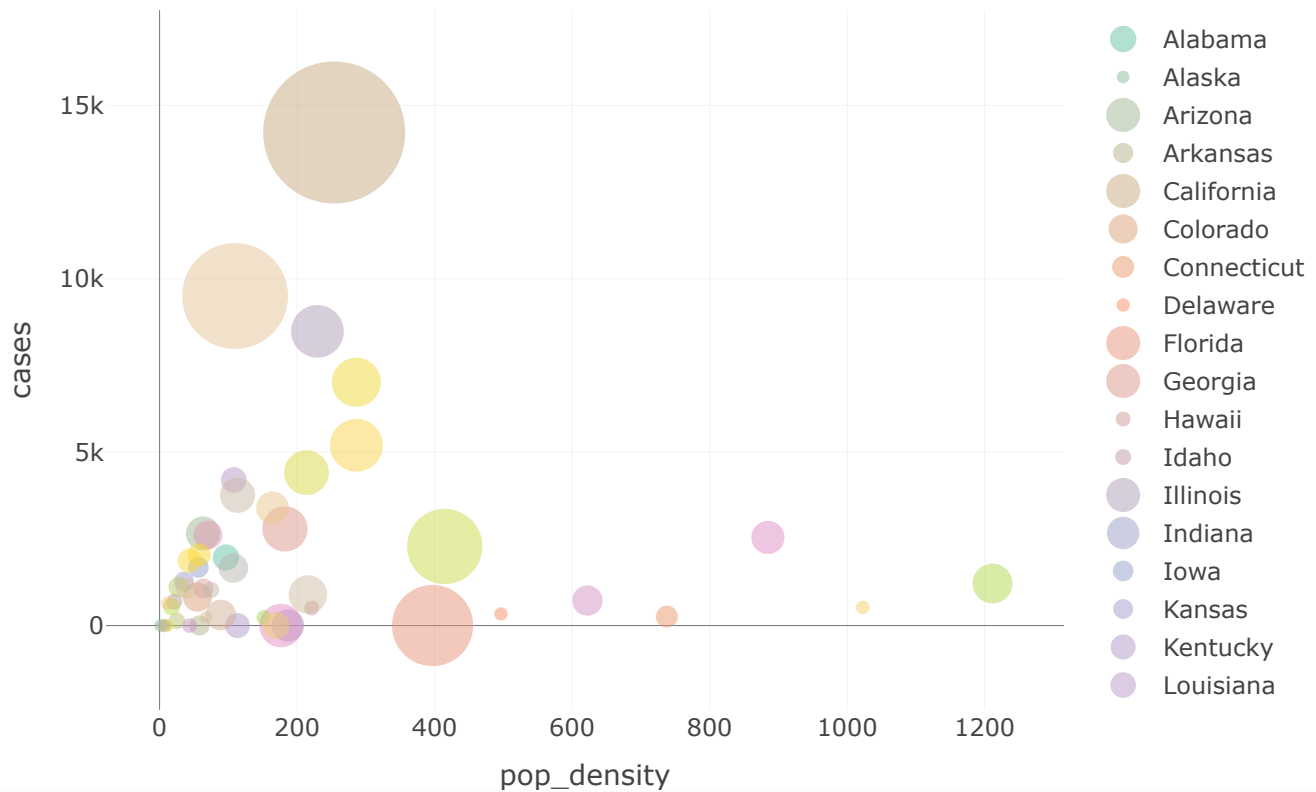
```
### FINISH CODE HERE
# pop_density vs. cases
cv_states_today |>
  plot_ly(x = ~pop_density, y = ~cases,
          type = 'scatter', mode = 'markers', color = ~state,
          size = ~population, sizes = c(5, 70), marker = list(sizemode='diameter', opacity=0.5))
```



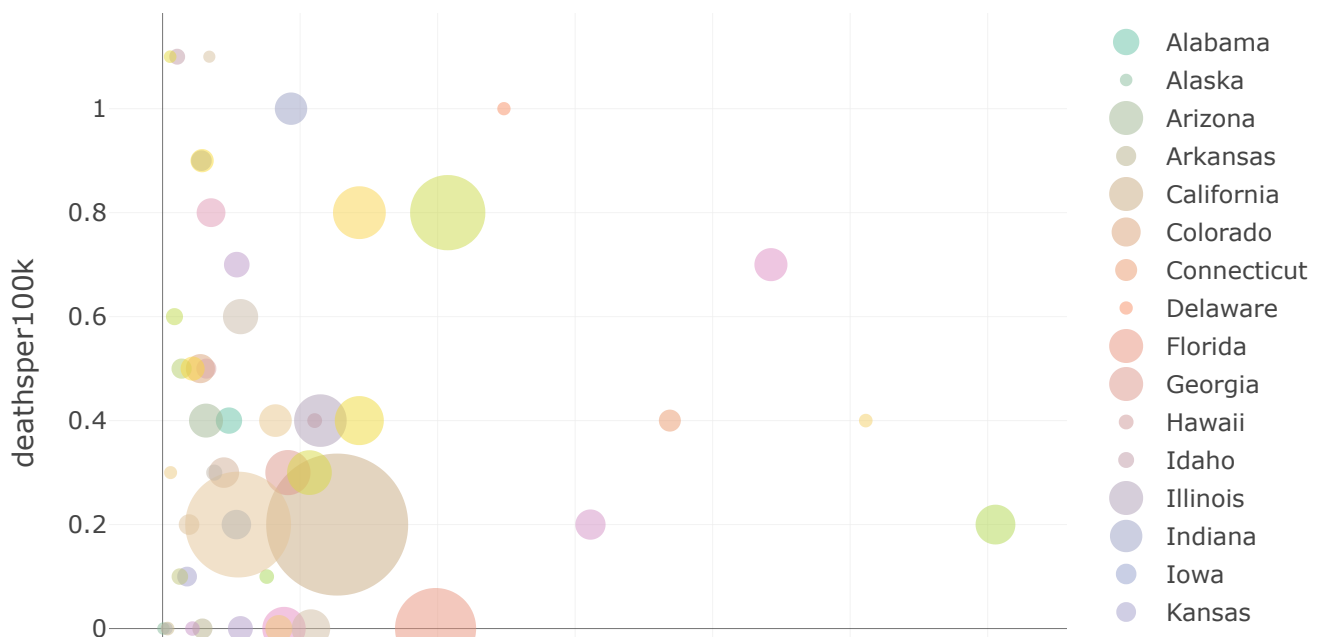
```
# filter out "District of Columbia"
cv_states_today_filter <- cv_states_today |> filter(state!="District of Columbia")

# pop_density vs. cases after filtering
cv_states_today_filter |>
  plot_ly(x = ~pop_density, y = ~cases,
```

```
type = 'scatter', mode = 'markers', color = ~state,
size = ~population, sizes = c(5, 70), marker = list(sizemode='diameter', opacity=0.5))
```



```
# pop_density vs. deathspers100k
cv_states_today_filter |>
  plot_ly(x = ~pop_density, y = ~deathspers100k,
    type = 'scatter', mode = 'markers', color = ~state,
    size = ~population, sizes = c(5, 70), marker = list(sizemode='diameter', opacity=0.5))
```

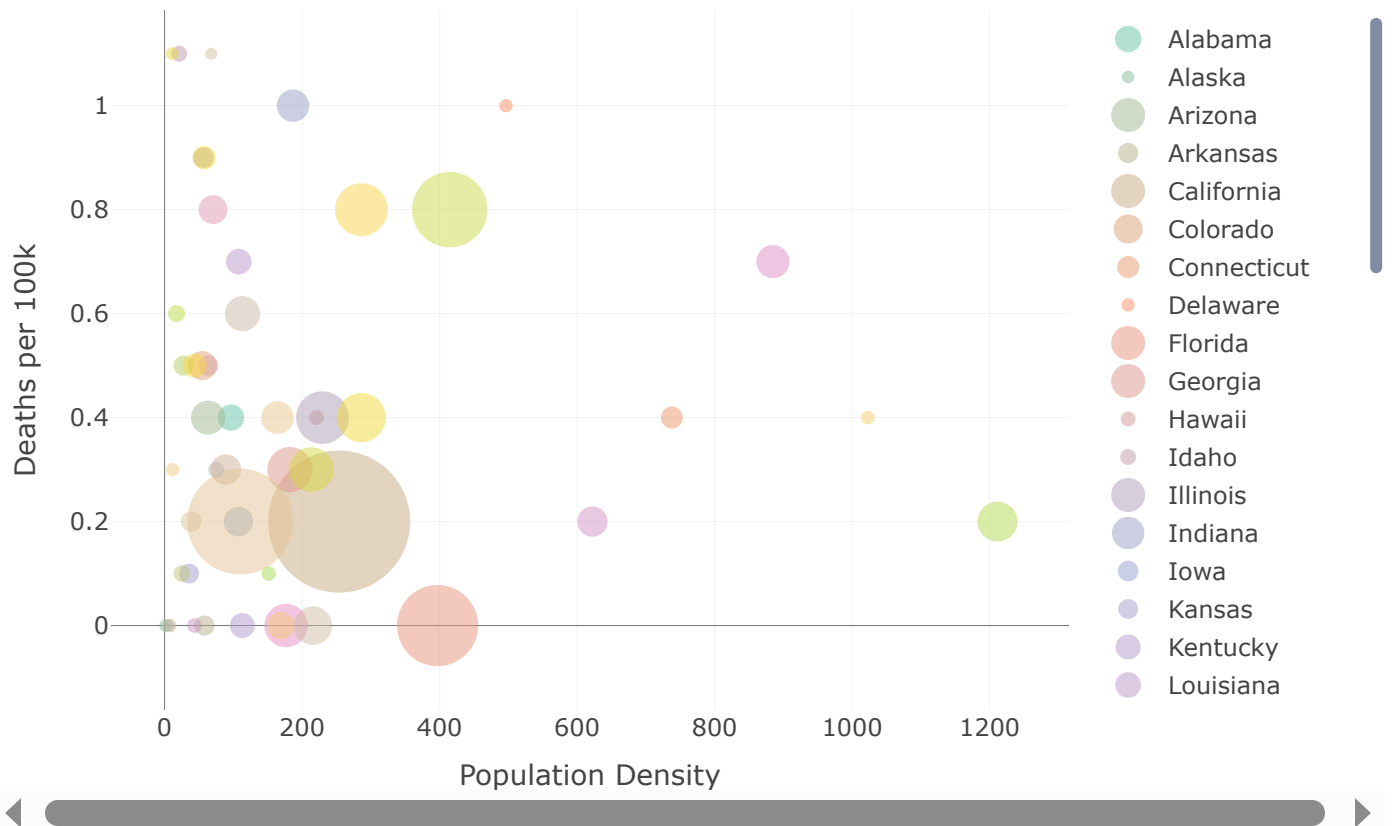






```
# Adding hoverinfo
cv_states_today_scatter <- cv_states_today_filter |>
  plot_ly(x = ~pop_density, y = ~deathsper100k,
    type = 'scatter', mode = 'markers', color = ~state,
    size = ~population, sizes = c(5, 70), marker = list(sizemode='diameter', opacity=0.5),
    hoverinfo = 'text',
    text = ~paste( paste(state, ":", sep=""), paste(" Cases per 100k: ", per100k, sep="") ,
      paste(" Deaths per 100k: ", deathsper100k, sep=""), sep = "<br>")) |>
  layout(title = "Population-normalized COVID-19 deaths (per 100k) vs. population density for US states",
    yaxis = list(title = "Deaths per 100k"), xaxis = list(title = "Population Density"),
    hovermode = "compare")
cv_states_today_scatter
```

Population-normalized COVID-19 deaths (per 100k) vs. population density for US states

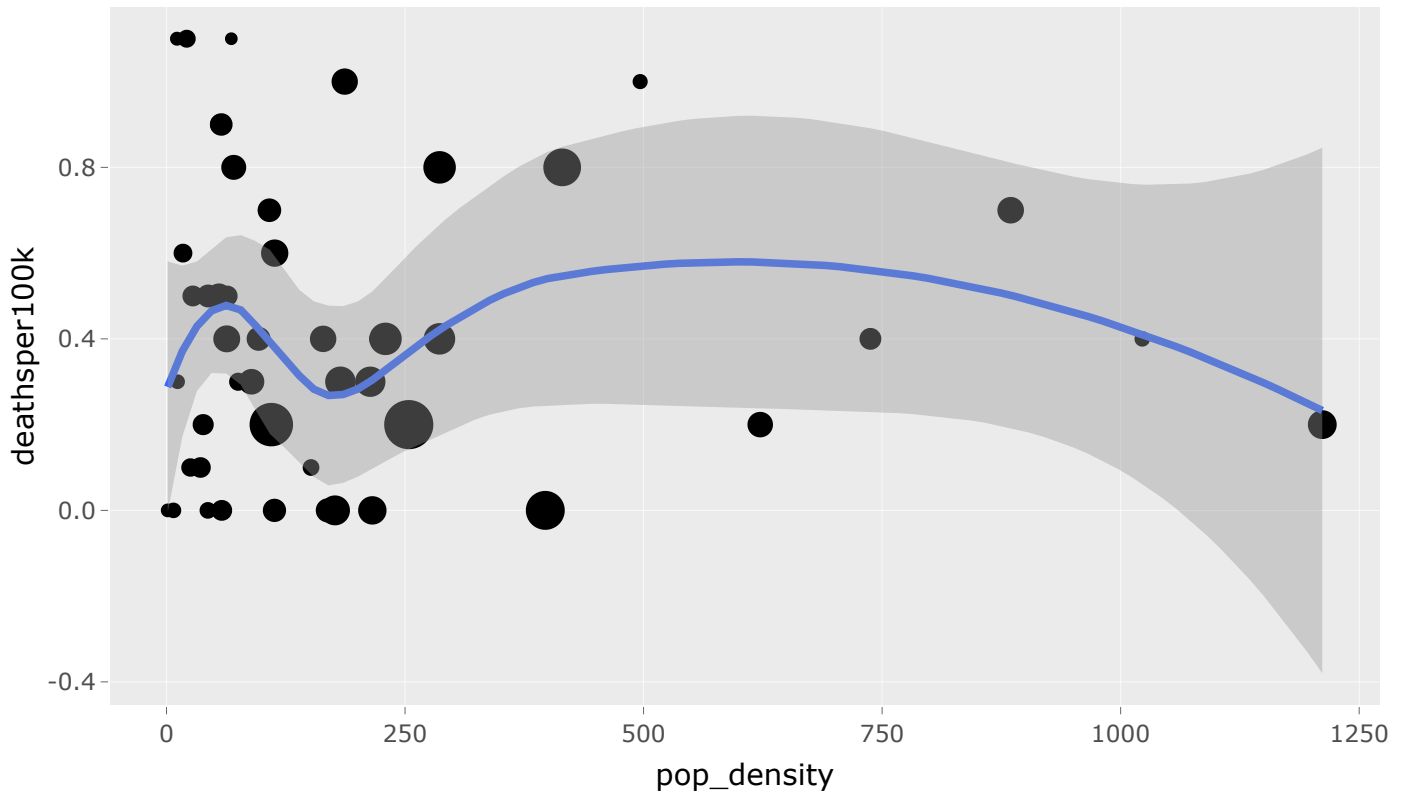


## 7. Explore scatterplot trend interactively using `ggplotly()` and `geom_smooth()`

- For `pop_density` vs. `deathsper100k` create a chart with the same variables using `ggplotly()`
- Explore the pattern between  $x$  and  $y$  using `geom_smooth()`

```
### FINISH CODE HERE
```

```
p <- ggplot(cv_states_today_scatter, aes(x=pop_density, y=deathsper100k, size=population)) + geom_
ggplotly(p)
```



## 8. Multiple line chart

- Create a line chart of the `naive_CFR` for all states over time using `plot_ly()`
  - Use the zoom and pan tools to inspect the `naive_CFR` for the states that had an increase in September.
- Create one more line chart, for Florida only, which shows `new_cases` and `new_deaths` together in one plot. Hint: look for an `add_*`()
  - Use `hoverinfo` to "eyeball" the approximate peak of deaths and peak of cases. What is the time delay between the peak of cases and the peak of deaths?

```
### FINISH CODE HERE
```

```
# Line chart for naive_CFR for all states over time using `plot_ly()
```

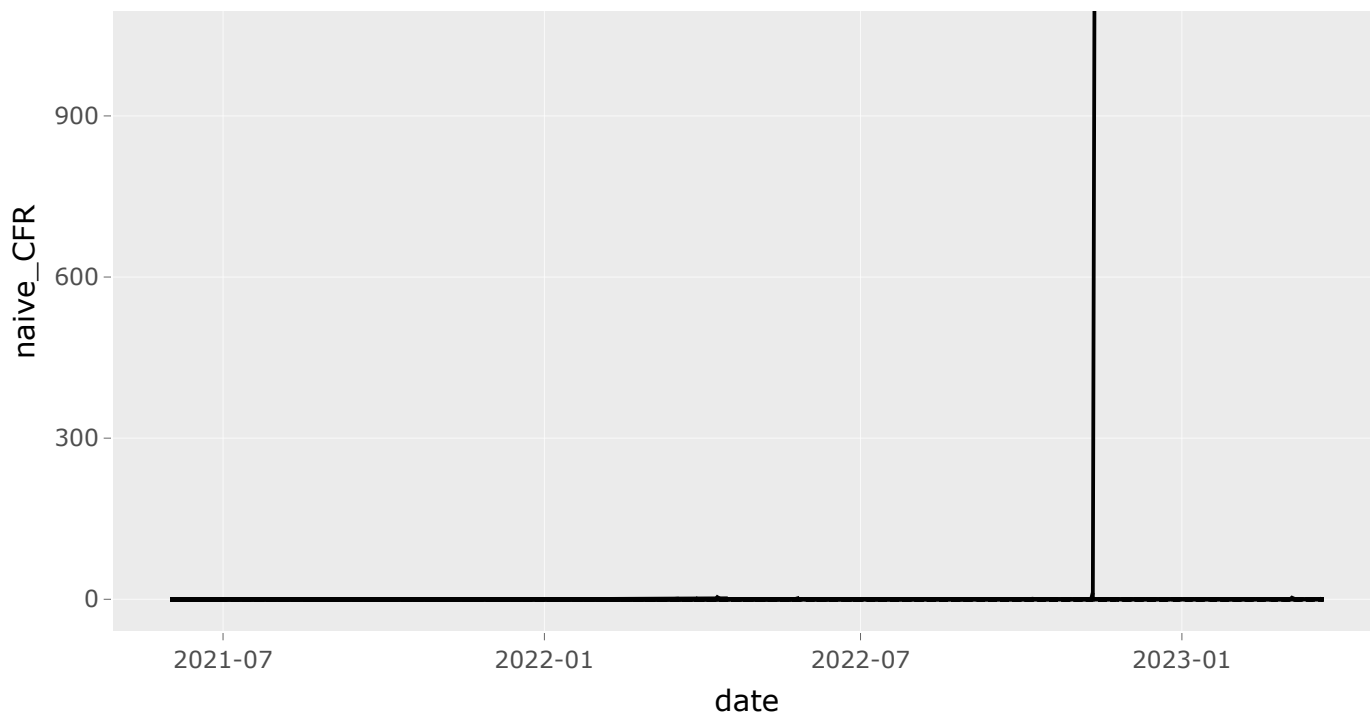
```
cv_states_cfr <- cv_states %>%
```

```
  filter(cv_states$naive_CFR < 1)
```

```
p <- ggplot(cv_states, aes(x = date, y=naive_CFR, group=state),) + geom_line()
```

```
ggplotly(p)
```

1200



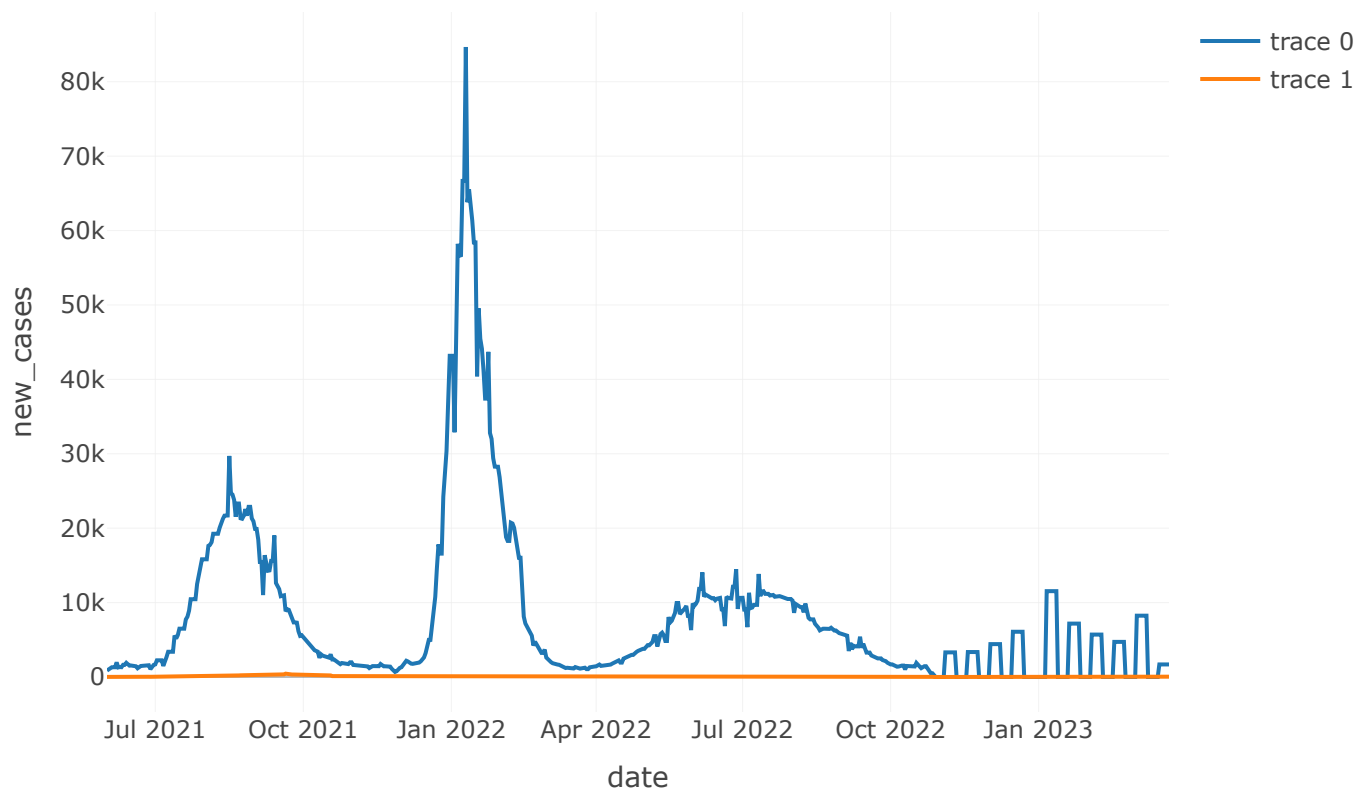
```
### FINISH CODE HERE
```

```
# Line chart for Florida showing new_cases and new_deaths together (two lines)
```

```
cv_states |> filter(state=="Florida") |>
```

```
  plot_ly(x = ~date, y = ~new_cases, type = "scatter", mode = "lines", hoverinfo = 'text') |>
```

```
  add_lines(x = ~date, y = ~new_deaths, type = "scatter", mode = "lines", hoverinfo = 'text')
```



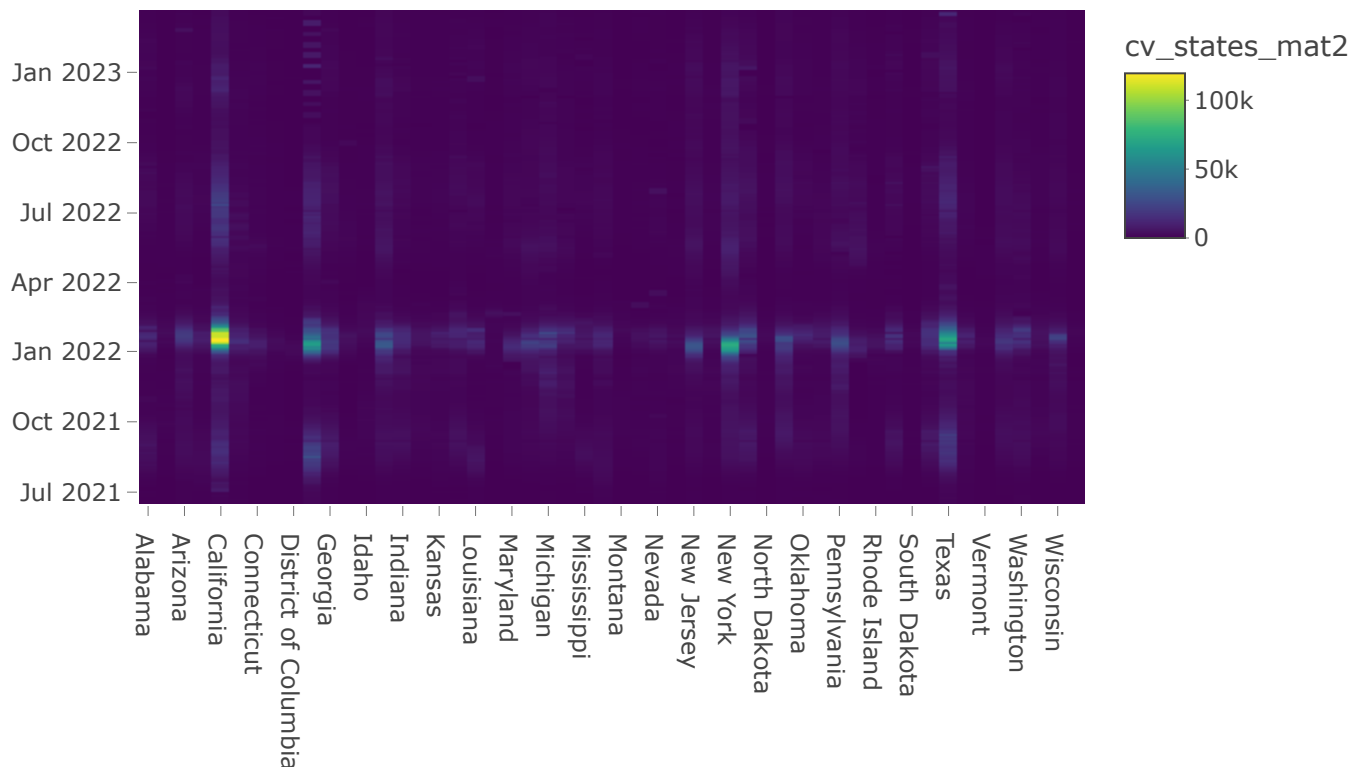
The peak of cases appears to be in February 2022, which is four months after the peak of deaths in October 2021.

## 9. Heatmaps

Create a heatmap to visualize `new_cases` for each state on each date greater than June 1st, 2021 - Start by mapping selected features in the dataframe into a matrix using the `tidyr` package function `pivot_wider()`, naming the rows and columns, as done in the lecture notes - Use `plot_ly()` to create a heatmap out of this matrix. Which states stand out? - Repeat with `newper100k` variable. Now which states stand out? - Create a second heatmap in which the pattern of `new_cases` for each state over time becomes more clear by filtering to only look at dates every two weeks

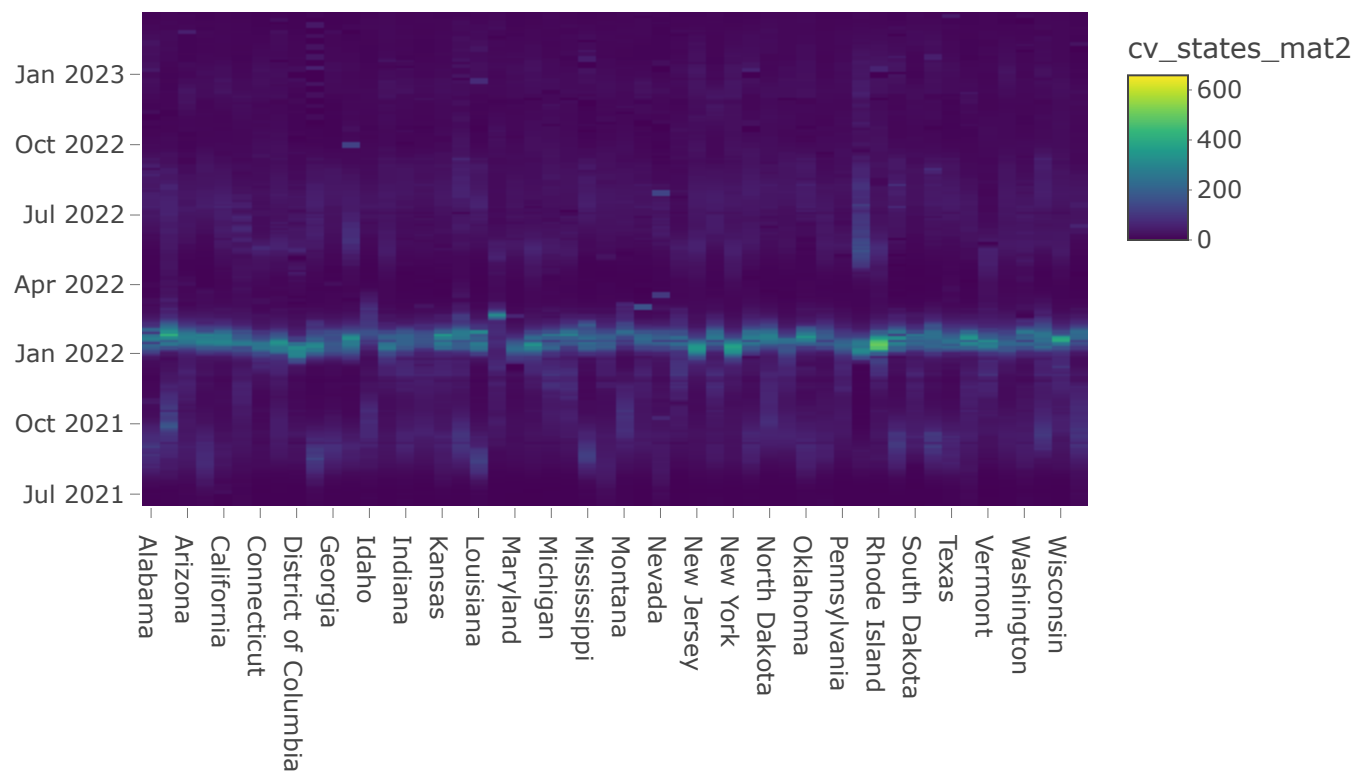
```
### FINISH CODE HERE
# Map state, date, and new_cases to a matrix
library(tidyr)
cv_states_mat <- cv_states |> select(state, date, new_cases) |> dplyr::filter(date>as.Date("2021-06-01"))
cv_states_mat2 <- as.data.frame(pivot_wider(cv_states_mat, names_from = state, values_from = new_cases))
rownames(cv_states_mat2) <- cv_states_mat2$date
cv_states_mat2$date <- NULL
cv_states_mat2 <- as.matrix(cv_states_mat2)

# Create a heatmap using plot_ly()
plot_ly(x=colnames(cv_states_mat2), y=rownames(cv_states_mat2),
        z=cv_states_mat2,
        type="heatmap",
        showscale=T)
```



```
# Repeat with newer100k
cv_states_mat <- cv_states |> select(state, date, newer100k) |> dplyr::filter(date>as.Date("2021-06-15"))
cv_states_mat2 <- as.data.frame(pivot_wider(cv_states_mat, names_from = state, values_from = newer100k))
rownames(cv_states_mat2) <- cv_states_mat2$date
cv_states_mat2$date <- NULL
cv_states_mat2 <- as.matrix(cv_states_mat2)

plot_ly(x=colnames(cv_states_mat2), y=rownames(cv_states_mat2),
        z=~cv_states_mat2,
        type="heatmap",
        showscale=T)
```

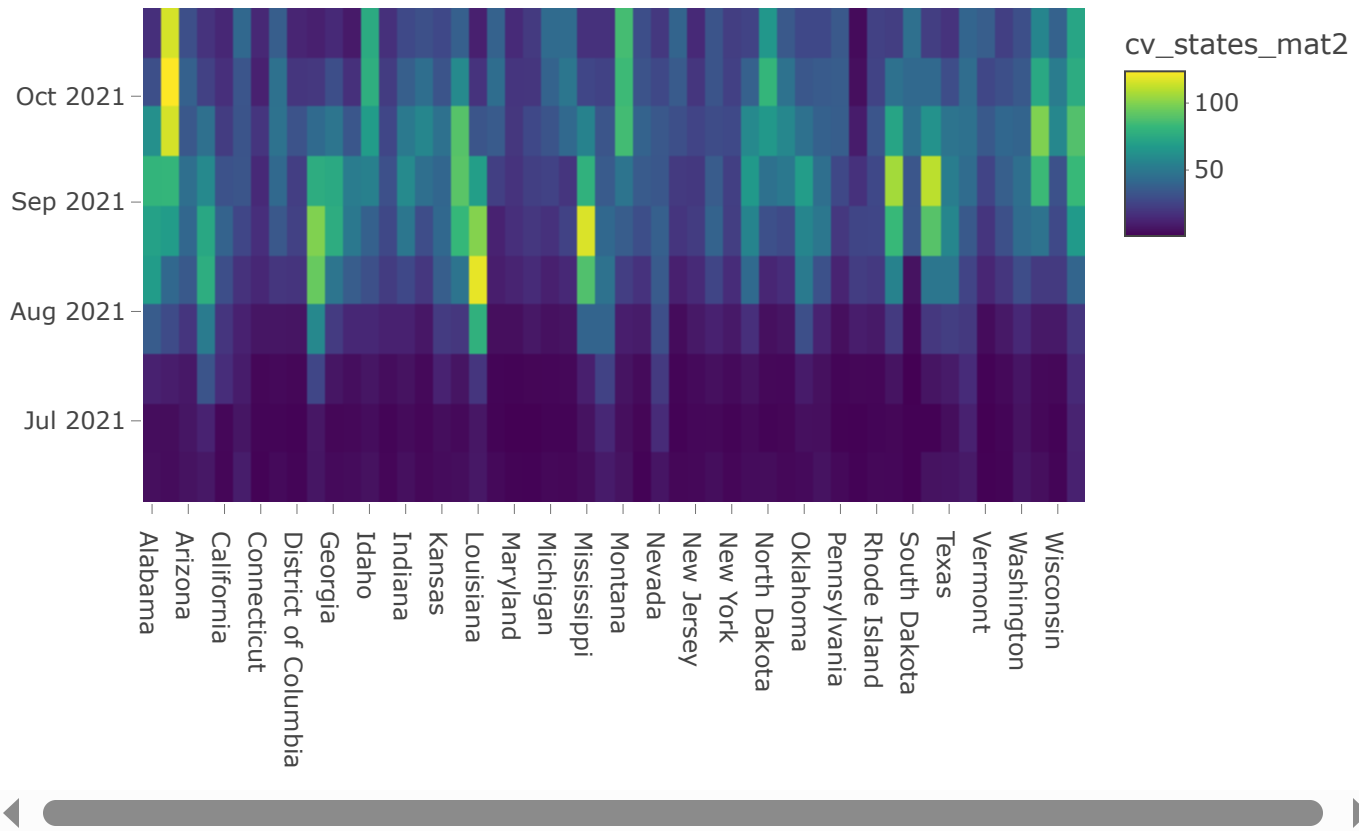


```
# Create a second heatmap after filtering to only include dates every other week
filter_dates <- seq(as.Date("2021-06-15"), as.Date("2021-11-01"), by=14)

cv_states_mat <- cv_states |> select(state, date, newer100k) |> filter(date %in% filter_dates)
cv_states_mat2 <- as.data.frame(pivot_wider(cv_states_mat, names_from = state, values_from = newer100k))
rownames(cv_states_mat2) <- cv_states_mat2$date
cv_states_mat2$date <- NULL
cv_states_mat2 <- as.matrix(cv_states_mat2)

# Create a heatmap using plot_ly()
plot_ly(x=colnames(cv_states_mat2), y=rownames(cv_states_mat2),
```

```
z=~cv_states_mat2,
type="heatmap",
showscale=TRUE)
```



## 10. Map

- Create a map to visualize the `naive_CFR` by state on October 15, 2021
- Compare with a map visualizing the `naive_CFR` by state on most recent date
- Create hover text to include state name, new cases per 100k, cases, and deaths
- Plot the two maps together using `subplot()`. Make sure the shading is for the same range of values (google is your friend for this)
- Describe the difference in the pattern of the CFR.

```
### For specified date
```

```
pick.date <- "2021-10-15"
```

```
# Extract the data for each state by its abbreviation
```

```
cv_per100 <- cv_states |> filter(date==pick.date) |> select(state, abb, newper100k, cases, deaths)
```

```
cv_per100$state_name <- cv_per100$state
```

```
cv_per100$state <- cv_per100$abb
```

```
cv_per100$abb <- NULL
```

```
# Create hover text
```

```

cv_per100$hover <- with(cv_per100, paste(state_name, '<br>', "Cases per 100k: ", newper100k, '<br>

# Set up mapping details
set_map_details <- list(
  scope = 'usa',
  projection = list(type = 'albers usa'),
  showlakes = TRUE,
  lakecolor = toRGB('white')
)

# Make sure both maps are on the same color scale
shadeLimit <- 125

# Create the map
fig <- plot_geo(cv_per100, locationmode = 'USA-states') |>
  add_trace(
    z = ~newper100k, text = ~hover, locations = ~state,
    color = ~newper100k, colors = 'Purples'
  )
fig <- fig |> colorbar(title = paste0("Cases per 100k: ", pick.date), limits = c(0,shadeLimit))
fig <- fig |> layout(
  title = paste('Cases per 100k by State as of ', pick.date, '<br>(Hover for value)'),
  geo = set_map_details
)
fig_pick.date <- fig

#####
### Map for today's date

# Extract the data for each state by its abbreviation
cv_per100 <- cv_states_today |> select(state, abb, newper100k, cases, deaths) # select data
cv_per100$state_name <- cv_per100$state
cv_per100$state <- cv_per100$abb
cv_per100$abb <- NULL

# Create hover text to include state name, new cases per 100k, cases, and deaths
cv_per100$hover <- with(cv_per100, paste(state.name, "<br>New cases per 100k", newper100k, "<br>C

# Set up mapping details
set_map_details <- list(
  scope = 'usa',
  projection = list(type = 'albers usa'),
  showlakes = TRUE,
  lakecolor = toRGB('white')
)

# Create the map
fig <- plot_geo(cv_per100, locationmode = 'USA-states') |>
  add_trace(
    z = ~newper100k, text = ~hover, locations = ~state,
    color = ~newper100k, colors = 'Purples'
  )

```

```
)  
fig <- fig |> colorbar(title = paste0("Cases per 100k: ", Sys.Date()), limits = c(0,shadeLimit))  
fig <- fig |> layout(  
  title = paste('Cases per 100k by State as of', Sys.Date(), '<br>(Hover for value)'),  
  geo = set_map_details  
)  
fig_Today <- fig  
  
### Plot together  
ggplotly(fig_pick.date, fig_Today, nrows = 2, margin = .05)
```

Cases per 100k by State as of 2021-10-15  
(Hover for value)

